

# Classification of Factual and Non-Factual data

Trisha Radhakrishnan  
BL.SC.U4AIE24152

Amrita School of Engineering  
Bengaluru, India

Srinithi Misha M  
BL.SC.U4AIE24149

Amrita School of Engineering  
Bengaluru, India

Navnita Krishnan  
BL.SC.U4AIE24134

Amrita School of Engineering  
Bengaluru, India

**Abstract**—The automatic classification of textual statements as factual or non-factual is a challenge in natural language processing with vast applications in misinformation detection, information retrieval, and knowledge base construction. So this work presents a comprehensive study of fact vs. non-fact classification using a combination of pre-trained and fine-tuned transformer-based text embeddings paired with traditional machine learning classifiers. We evaluate seven embedding strategies — TF-IDF, Word2Vec, SBERT, BERT, T5, XLM-RoBERTa, and UniXcoder — alongside their fine-tuned counterparts, across seven classifiers: K-Nearest Neighbors, Decision Tree, Logistic Regression, Support Vector Machine (SVM), XGBoost, LightGBM, and CatBoost. To address class imbalance and length-induced bias in the dataset, we apply length-stratified resampling prior to training. Model interpretability is examined using LIME. The top-performing configuration is SVM combined with T5 pre-trained embeddings, demonstrating the effectiveness of encoder-based semantic representations for binary text classification tasks.

## I. INTRODUCTION

The growth of digital information has made it essential to tell difference in factual and non-factual content. Fact-checking systems, automated knowledge graph validation, and misinformation detection pipelines all need good ways to classify statements by their factual nature. While human-labeled fact-checking exists, it cannot keep up with the overwhelming amount of information produced every day. Machine learning methods, especially transformer-based models, provide a promising route to automation.

Traditional NLP methods for text classification usually depend on basic features like bag-of-words or TF-IDF representations. Although these methods are efficient, they struggle to capture deeper meanings and context. The rise of large pre-trained language models, such as BERT, T5, and XLM-RoBERTa, has notably transformed text representation. These models deliver contextually rich embeddings that convey subtle linguistic details.

In this study, we focus on the task of classifying texts as factual or non-factual using a dataset of 1,300 labeled documents(sentences). One major challenge we tackle is the difference in length between fact and non-fact samples; facts are typically shorter, while non-facts are usually longer. To handle this issue, we apply length-stratified resampling. We then assess various embedding strategies, ranging from classical TF-IDF and Word2Vec to fine-tuned transformer-based representations, each coupled with a selection of standard

machine learning classifiers. We analyze the best-performing models using LIME for token-level interpretability, helping us understand which words influence classification choices.

The main contributions of this work is:

- systematic comparison of pre-trained and fine-tuned transformer embeddings for fact classification.
- A length-stratified resampling strategy to reduce document-length bias.
- An analysis of seven classifiers across eleven embedding configurations.
- A LIME-based interpretability study of the top five model-embedding combinations.

## II. LITERATURE REVIEW

### A. Fact vs. Non-Fact Classification in News Articles

The most relevant work to our proposed approach is by Sahu and Majumdar [1] where the problem of fact vs. non-fact detection was studied in the context of news articles. Their approach used segmentation at sentence and quotation level along with analysis of multiple features such as n-grams, part-of-speech (POS) tagging, named entity types, AutoSlog-TS information extraction patterns, Turney’s subjective patterns, sentiment scores, and positional features. The classifiers used were Linear SVMs using LIBLINEAR library. They experimented with two datasets, the MPQA Opinion Corpus and the Signal Media One-Million News Articles Dataset. Their observations indicated that n-gram features give consistent results on both datasets, while general features like POS tagging and entity types are inconsistent in terms of dataset quality. In particular, they showed that the presence of a good quality, task-specific annotation scheme was critical for achieving reliable results in fact classification. These works provide fundamental motivations for the proposed approach where we replace engineered features with dense representation from transformers. They have also written an M.Tech thesis companion paper [2] giving an extensive review of previous work done in the field of subjectivity and objectivity classification in relation to fact detection.

### B. Detection of Fake News Using Machine Learning

Another highly relevant research area involves fake news detection, i.e., the process of detecting content that is either

false or misleading. For instance, Probiez, Stefański, and Kozak [3] introduced an efficient system for detecting fake news through NLP and ensemble classifiers on the ISOT Fake News Dataset. SVM, CART, Random Forest, AdaBoost, and Bagging were evaluated in the study using TF-IDF and term frequency representations. SVM showed the highest level of accuracy (0.9419) when classifying based on titles, whereas ensemble techniques like Random Forest gave a more favorable speed/accuracy ratio. The use of full articles in training resulted in almost perfect accuracy rates (up to 0.9964 when using Bagging), which emphasized the importance of text content in the distinction between true and fake news. Sharifani et al. [4] provided a general survey on NLP and machine learning approaches for fake news detection. They used Passive Aggressive, Naïve Bayes, and SVM classifiers on Kaggle and Signal Media datasets. The authors' Passive Aggressive classifier had a high accuracy rate of 93 percent, surpassing SVM, 84 percent and Naïve Bayes, 85 percent.

### C. Verification of Scientific Claims

The paper by Wadden et al. [5] proposed the use of SCIFACT, a data set containing 1,409 scientific claims written by experts and accompanied by biomedical research abstracts that were annotated as either SUPPORTS, REFUTES, or NOINFO. The pipeline model developed by the researchers, VERISCI, included an abstract retriever based on TF-IDF, a rationale selector using BERT, and a label predictor based on RoBERTa-large, where the model was first pre-trained on the general FEVER data set and then fine-tuned on SCIFACT. It is important to note that the authors stressed the importance of fine-tuning for specific domains because models trained on the FEVER data set and then fine-tuned zero-shot for SCIFACT performed poorly.

### D. Evidence-Based Fact-Checking

Alhindi, Petridis, and Muresan [6] augmented the LIAR dataset (claims from POLITIFACT) with manually created justifications, resulting in LIAR-PLUS. They demonstrated that jointly modeling the justification and the claim resulted in substantial gains for all models — Logistic Regression, SVM, and BiLSTM, in both binary and six-class classifications. In the six-class task, adding justifications resulted in an improvement in BiLSTM F1 score from 0.23 to 0.34 and LR F1 score from 0.25 to 0.37. The dual-architecture BiLSTM model they designed to jointly model the claim and justification separately prior to fusion is analogous to current approaches used by transformers.

### E. Bias in Fact Verification Datasets

Schuster et al. studied the problem of bias in the FEVER fact verification dataset and demonstrated that models can easily predict the label based on linguistic features of claims without analyzing evidence. In their research, the authors

found that annotation artifacts generated unintended correlations between linguistic patterns and the target label, leading to the use of shortcuts by models instead of proper reasoning. In order to solve the problem, the researchers created a symmetric evaluation set comprising balanced claim-evidence pairs in such a way as to avoid any bias exploitation. The researchers also suggested an approach for debiasing that is based on re-weighting regularization. This paper is related to our work in terms of highlighting the need to diminish surface and distributional biases in classification problems.

## III. SOTA

The Table I shows the comparison between our approach and some of the important results obtained by researchers in the relevant literature. The datasets and tasks vary for each research work.

## IV. DATASET AND PREPROCESSING

### A. Dataset

The dataset used in this study is a custom-collected corpus stored as data1300.csv. It consists of 1,300 labeled text samples. Each sample includes a document field (Doc) and a binary label (Label), which can be "Fact" or "Non-Fact". The dataset was created to include short-form statements from various domains.

### B. Class and Length Distribution

An initial analysis of the dataset showed a length difference between classes. Fact samples tend to be shorter, while Non-Fact samples are generally longer. This length imbalance can lead classifiers to focus on document length instead of the actual content. To visualize this, we plotted the word-count distributions for both classes before and after resampling, using histograms with 30 bins in different word-count ranges.

### C. Length-Stratified Resampling

To reduce length-related bias, we used a length-stratified resampling method. We categorized documents into six length groups: 0-5, 6-10, 11-15, 16-20, 21-30, and 30+ words. In each group, Fact samples were down-sampled to match the number of Non-Fact samples. This helps keep the class distribution balanced within each length category, preventing classifiers from relying on length as a key feature. The final resampled dataset is balanced in both class and length.

### D. Data split

D. Train-Test Split The resampled dataset was split into training, 80 percent and test, 20 percent sets using a stratified method with a fixed random seed of 42 to ensure that the same data will be reproduced everytime the code is run. Label encoding maps "Fact" to 1 and "Non-Fact" to 0.

TABLE I  
COMPARISON OF EXISTING APPROACHES WITH OUR WORK

| Reference             | Task                               | Embedding/ Features             | Best Classifier     | Best Test Accuracy  | Best Test F1 Accuracy |
|-----------------------|------------------------------------|---------------------------------|---------------------|---------------------|-----------------------|
| Sahu & Majumdar [1,2] | Fact vs. Non-Fact in news          | N-grams, POS, Entity, Sentiment | Linear SVM          | 0.72 (Signal Media) | 0.77 (Fact F1)        |
| Probierz et al. [3]   | Fake news detection (title)        | TF-IDF                          | SVM                 | 0.9419              | 0.9445                |
| Probierz et al. [3]   | Fake news detection (full text)    | TF-IDF                          | Bagging             | 0.9964              | 0.9965                |
| Sharifani et al. [4]  | Fake news detection                | TF-IDF + CountVectorizer        | Passive Aggressive  | 0.93                | –                     |
| Alhindi et al. [6]    | Claim veracity (LIAR-PLUS, binary) | Unigrams + Justification        | BiLSTM (SJ)         | 0.68                | –                     |
| Alhindi et al. [6]    | Claim veracity (LIAR-PLUS, 6-way)  | Unigrams + Justification        | LR (S+MJ)           | –                   | 0.38                  |
| Schuster et al. [7]   | Fact verification (FEVER)          | BERT fine-tuned                 | BERT + Re-weighting | 0.846               | –                     |
| Wadden et al. [5]     | Scientific claim verification      | RoBERTa-large (fine-tuned)      | VERISCI Pipeline    | –                   | 0.727 (Abstract F1)   |
| Our Work              | Fact vs. Non-Fact classification   | T5 pre-trained (mean pool)      | SVM                 | 0.958               | 0.958                 |
| Our Work              | Fact vs. Non-Fact classification   | SBERT                           | SVM                 | 0.9496              | 0.9495                |
| Our Work              | Fact vs. Non-Fact classification   | BERT-FT (fine-tuned)            | SVM                 | 0.9496              | 0.9495                |

## V. METHODOLOGY

### A. Text Embedding Strategies

We evaluated eleven embedding configurations, which were divided into pre-trained and fine-tuned categories.

Pre-trained Embeddings:

TF-IDF: A sparse bag-of-words representation using unigrams and bigrams, with a maximum of 1,000 features, removal of English stop words, and a minimum document frequency of 1 percent.

Word2Vec: A dense 100-dimensional embedding trained on the training corpus using the CBOW method (window size 5, min count 1). Each document is represented as the mean of its token vectors.

SBERT (all-MiniLM-L6-v2): Sentence embeddings from the Sentence-BERT framework, which generates fixed-size dense vectors optimized for semantic similarity.

BERT (bert-base-uncased): The [CLS] token representation taken from the final hidden layer of BERT, using CLS pooling over tokenized input (max length 128).

T5 (t5-base): Contextual embeddings obtained from the T5 encoder using attention-masked mean pooling, which calculates a weighted average of token representations while excluding padding tokens.

XLm-RoBERTa (xlm-roberta-base): CLS-token embeddings from the cross-lingual RoBERTa model, suitable for multilingual classification tasks.

UniXcoder (microsoft/unixcoder-base): CLS-token embeddings from Microsoft’s UniXcoder, a unified cross-modal pre-

trained model created for both code and natural language understanding.

Fine tuned embeddings :

Fine-tuned versions were also obtained for BERT, T5, XLm-RoBERTa and UniXcoder. We adapted each model with a classification head (linear layer mapping hidden size  $\rightarrow$  2 classes) on top of the base encoder. AdamW optimizer with linear learning rate warmup schedule, batch size of 16 and learning rate of  $2 \times 10^{-5}$ . Training was done for 3 epochs. We extracted the pooled hidden-state representations (CLS for BERT/XLm/UniXcoder; attention-masked mean for T5) from the fine-tuned encoder and saved them as dense numpy arrays to train the downstream classifier.

### B. Classifiers

For each embedding, we tested seven classical machine learning classifiers:

- K-Nearest Neighbors (KNN): hyperparameter search on k 3, 5, 7, 9, 11, 13, 15, distance metrics (Euclidean, Manhattan) and weighting (uniform, distance).
- Decision Tree: Randomized search over max depth, min samples split and min samples leaf.
- Logistic Regression: L2 regularization, liblinear solver, class balanced weighting, C sampled from a log-uniform distribution over [0.01, 100].
- Support Vector Machine (SVM) With class balanced weighting, randomized search over kernels (RBF, linear, polynomial, sigmoid), regularization C and gamma.
- XGBoost: With search over number of estimators, tree depth, learning rate, subsample and column subsampling

- LightGBM: Search over equivalent parameters, class balance.
- CatBoost: with search on number of iterations, depth, learning rate, L2 regularization and automatic class weight strategies.

All classifiers were optimized using RandomizedSearchCV with 10 iterations, 3-fold cross-validation and macro F1 as scoring objective.

### C. Metrics for Evaluation

The models were evaluated on the training set and the test set with two metrics: accuracy and macro F1 score. Given the binary classes and the resampled balanced distribution, the main metric is the macro F1 score.

### D. t-SNE Visualization

To qualitatively assess the separability of the embedding spaces, we applied t-SNE (t-distributed Stochastic Neighbor Embedding) to all saved embedding matrices. For each embedding, perplexity was set to  $\min(30, n_l)$  and projected into 2 dimensions. The scatter plots obtained show visually how well each embedding organizes the feature space with respect to classification.

### E. LIME for model interpretability

We analyzed the top five performing model–embedding combinations using LIME (Local Interpretable Model-Agnostic Explanations). LIME injects noise into the input and measures the impact on the classifier’s output to assign weight values to each token.

In each of the top-5 classifiers, `predict_proba` functions were built by passing the text data via the embedding model into the trained classifier.

LIME was performed using the parameters `bow=False`, which preserves word order and is more suitable for transformer embeddings, `num_features=10`, and `num_samples=100`.

Tokens contributing to predictions were identified by combining word-level importance scores across all test samples.

## VI. EXPERIMENTS AND RESULTS

All seven classifiers were applied to all eleven embedding settings. The performance was evaluated for both the training and test datasets, with macro F1 being the main factor for ranking the models. The superiority of SVM ranking emphasizes the applicability of this model in high-dimensional dense embedding space. T5 Encoder’s attention masked mean pooling technique demonstrates great effectiveness, surpassing even the fine-tuned version in the pre-trained condition, implying the transferability of semantic representation generated during pre-training.

TABLE II  
PERFORMANCE COMPARISON OF TRADITIONAL TRANSFORMER-BASED EMBEDDINGS

| Model               | Embedding | Train Acc | Train F1 | Test Acc | Test F1 |
|---------------------|-----------|-----------|----------|----------|---------|
| KNN                 | TF-IDF    | 0.7703    | 0.7673   | 0.6891   | 0.6841  |
| Decision Tree       | TF-IDF    | 0.6807    | 0.6374   | 0.6555   | 0.6116  |
| Logistic Regression | TF-IDF    | 0.7640    | 0.7607   | 0.7185   | 0.7130  |
| SVM                 | TF-IDF    | 0.7724    | 0.7698   | 0.7101   | 0.7051  |
| XGBoost             | TF-IDF    | 0.6449    | 0.5933   | 0.6387   | 0.5900  |
| CatBoost            | TF-IDF    | 0.7987    | 0.7967   | 0.7185   | 0.7142  |
| LightGBM            | TF-IDF    | 0.7060    | 0.6896   | 0.6975   | 0.6811  |
| KNN                 | Word2Vec  | 1.0000    | 1.0000   | 0.6597   | 0.6579  |
| Decision Tree       | Word2Vec  | 1.0000    | 1.0000   | 0.6050   | 0.6046  |
| Logistic Regression | Word2Vec  | 0.7597    | 0.7597   | 0.7143   | 0.7103  |
| SVM                 | Word2Vec  | 0.8430    | 0.8430   | 0.7983   | 0.7976  |
| XGBoost             | Word2Vec  | 0.9979    | 0.9979   | 0.6933   | 0.6932  |
| CatBoost            | Word2Vec  | 0.9779    | 0.9778   | 0.7017   | 0.7005  |
| LightGBM            | Word2Vec  | 1.0000    | 1.0000   | 0.6933   | 0.6926  |
| KNN                 | SBERT     | 1.0000    | 1.0000   | 0.8992   | 0.8992  |
| Decision Tree       | SBERT     | 0.9989    | 0.9989   | 0.7185   | 0.7185  |
| Logistic Regression | SBERT     | 0.9410    | 0.9409   | 0.9412   | 0.9411  |
| SVM                 | SBERT     | 1.0000    | 1.0000   | 0.9496   | 0.9495  |
| XGBoost             | SBERT     | 0.9968    | 0.9968   | 0.8824   | 0.8819  |
| CatBoost            | SBERT     | 1.0000    | 1.0000   | 0.9160   | 0.9159  |
| LightGBM            | SBERT     | 0.9926    | 0.9926   | 0.9118   | 0.9118  |
| KNN                 | BERT      | 1.0000    | 1.0000   | 0.9118   | 0.9116  |
| Decision Tree       | BERT      | 0.9389    | 0.9388   | 0.7857   | 0.7854  |
| Logistic Regression | BERT      | 1.0000    | 1.0000   | 0.9370   | 0.9369  |
| SVM                 | BERT      | 0.9947    | 0.9947   | 0.9454   | 0.9454  |
| XGBoost             | BERT      | 1.0000    | 1.0000   | 0.9286   | 0.9286  |
| CatBoost            | BERT      | 1.0000    | 1.0000   | 0.9118   | 0.9116  |
| LightGBM            | BERT      | 1.0000    | 1.0000   | 0.9412   | 0.9411  |
| KNN                 | T5        | 1.0000    | 1.0000   | 0.9160   | 0.9160  |
| Decision Tree       | T5        | 0.9389    | 0.9388   | 0.8487   | 0.8487  |
| Logistic Regression | T5        | 0.9895    | 0.9894   | 0.9454   | 0.9453  |
| SVM                 | T5        | 1.0000    | 1.0000   | 0.9580   | 0.9580  |
| XGBoost             | T5        | 1.0000    | 1.0000   | 0.9328   | 0.9328  |
| CatBoost            | T5        | 1.0000    | 1.0000   | 0.9412   | 0.9411  |
| LightGBM            | T5        | 1.0000    | 1.0000   | 0.9454   | 0.9453  |
| KNN                 | XLM       | 0.9199    | 0.9193   | 0.8487   | 0.8481  |
| Decision Tree       | XLM       | 0.8588    | 0.8579   | 0.7353   | 0.7318  |
| Logistic Regression | XLM       | 0.9874    | 0.9873   | 0.9202   | 0.9200  |
| SVM                 | XLM       | 0.9842    | 0.9842   | 0.9160   | 0.9157  |
| XGBoost             | XLM       | 1.0000    | 1.0000   | 0.9076   | 0.9076  |
| CatBoost            | XLM       | 1.0000    | 1.0000   | 0.8908   | 0.8906  |
| LightGBM            | XLM       | 1.0000    | 1.0000   | 0.9160   | 0.9158  |
| KNN                 | UniXcoder | 0.9231    | 0.9224   | 0.8193   | 0.8166  |
| Decision Tree       | UniXcoder | 0.8809    | 0.8801   | 0.7017   | 0.7008  |
| Logistic Regression | UniXcoder | 0.9779    | 0.9779   | 0.8824   | 0.8821  |
| SVM                 | UniXcoder | 0.9989    | 0.9989   | 0.8950   | 0.8945  |
| XGBoost             | UniXcoder | 1.0000    | 1.0000   | 0.8697   | 0.8695  |
| CatBoost            | UniXcoder | 1.0000    | 1.0000   | 0.8571   | 0.8565  |
| LightGBM            | UniXcoder | 1.0000    | 1.0000   | 0.8782   | 0.8778  |

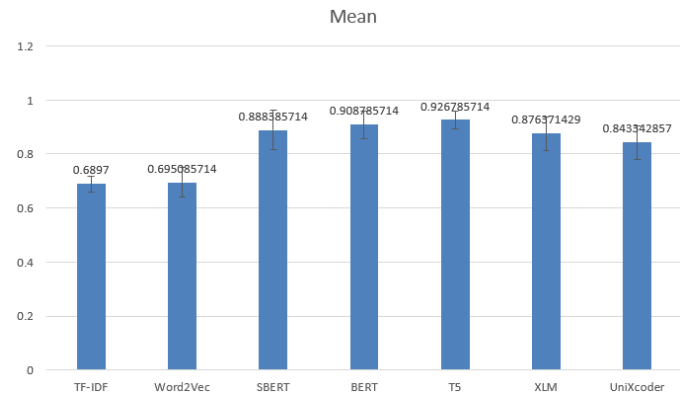


Fig. 1. Mean performance of pretrained models

### A. Fine-tuned

Fine-tuning improved performance for BERT and XLM-RoBERTa, particularly in macro F1 score. However, for T5, the pre-trained embeddings yielded competitive results without the additional training overhead, suggesting that T5’s encoder already produces semantically well-structured representations for this task. UniXcoder, while primarily designed for code understanding, showed reasonable performance, indicating some transferability to natural language classification.

TABLE III  
PERFORMANCE COMPARISON OF FINE UNED EMBEDDINGS

| Model | Embed.  | Tr Acc | Tr F1  | Te Acc | Te F1  |
|-------|---------|--------|--------|--------|--------|
| KNN   | BERT-FT | 0.9368 | 0.9367 | 0.9370 | 0.9369 |
| DT    | BERT-FT | 0.9916 | 0.9916 | 0.8529 | 0.8522 |
| LR    | BERT-FT | 0.9673 | 0.9673 | 0.9328 | 0.9328 |
| SVM   | BERT-FT | 1.0000 | 1.0000 | 0.9496 | 0.9495 |
| XGB   | BERT-FT | 0.9621 | 0.9620 | 0.8908 | 0.8907 |
| CatB  | BERT-FT | 0.9842 | 0.9842 | 0.9076 | 0.9074 |
| LGBM  | BERT-FT | 1.0000 | 1.0000 | 0.9076 | 0.9072 |
| KNN   | T5-FT   | 1.0000 | 1.0000 | 0.8992 | 0.8992 |
| DT    | T5-FT   | 1.0000 | 1.0000 | 0.8487 | 0.8487 |
| LR    | T5-FT   | 0.9916 | 0.9916 | 0.9454 | 0.9453 |
| SVM   | T5-FT   | 1.0000 | 1.0000 | 0.9538 | 0.9537 |
| XGB   | T5-FT   | 0.9905 | 0.9905 | 0.9286 | 0.9286 |
| CatB  | T5-FT   | 1.0000 | 1.0000 | 0.9244 | 0.9244 |
| LGBM  | T5-FT   | 1.0000 | 1.0000 | 0.9370 | 0.9370 |
| KNN   | XLM-FT  | 1.0000 | 1.0000 | 0.8739 | 0.8739 |
| DT    | XLM-FT  | 0.9821 | 0.9821 | 0.8529 | 0.8528 |
| LR    | XLM-FT  | 1.0000 | 1.0000 | 0.9454 | 0.9453 |
| SVM   | XLM-FT  | 1.0000 | 1.0000 | 0.9244 | 0.9241 |
| XGB   | XLM-FT  | 1.0000 | 1.0000 | 0.8950 | 0.8949 |
| CatB  | XLM-FT  | 1.0000 | 1.0000 | 0.9076 | 0.9075 |
| LGBM  | XLM-FT  | 1.0000 | 1.0000 | 0.8992 | 0.8990 |
| KNN   | UniX-FT | 1.0000 | 1.0000 | 0.8992 | 0.8989 |
| DT    | UniX-FT | 0.9663 | 0.9662 | 0.8109 | 0.8094 |
| LR    | UniX-FT | 0.9747 | 0.9747 | 0.9034 | 0.9031 |
| SVM   | UniX-FT | 1.0000 | 1.0000 | 0.8824 | 0.8821 |
| XGB   | UniX-FT | 1.0000 | 1.0000 | 0.8824 | 0.8821 |
| CatB  | UniX-FT | 1.0000 | 1.0000 | 0.9076 | 0.9073 |
| LGBM  | UniX-FT | 0.9958 | 0.9958 | 0.8655 | 0.8649 |

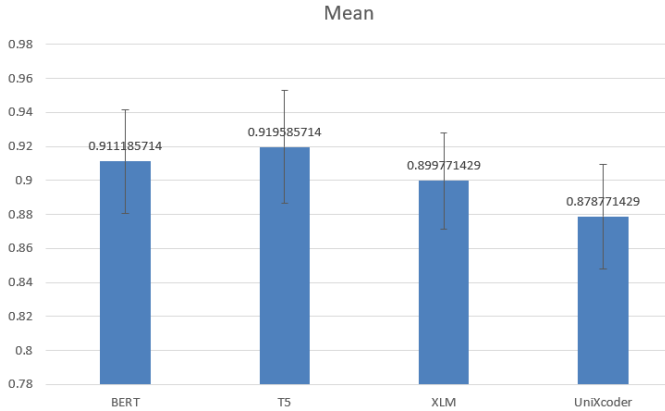


Fig. 2. Mean performance of finetuned

### B. t-SNE Analysis

The t-SNE plots of the embedding space show that the embeddings of T5 and SBERT have more class separation than TF-IDF and Word2Vec embeddings. It is observed that fine-tuned embeddings have better within-class clustering than their pre-trained versions, which can be attributed to their superior classification results.

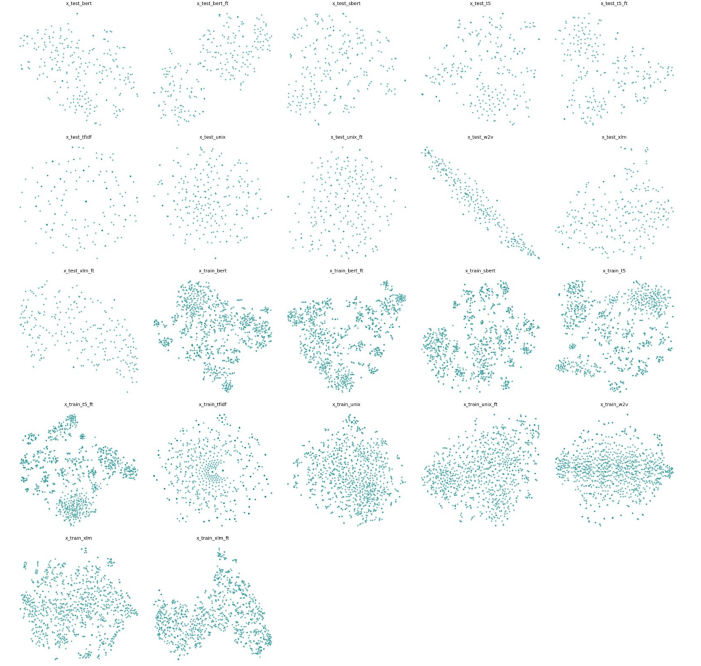


Fig. 3. t-SNE

### C. Fine-tuned End-to-End Sequence Classification (Transformer Models)

In addition to the above experiments, in Phase 8, we tested transformer models that have been fine-tuned end-to-end for classification purposes, namely BERT, Flan-T5 (google/flan-t5-base), XLM-RoBERTa, and UniXcoder. All models were trained for 3 epochs using a batch size of 16 and learning rate of  $2 \times 10^{-5}$  (except for T5 whose learning rate is  $1 \times 10^{-4}$ ). The results are presented

### D. LIME Interpretability Analysis

LIME analyses performed for the top-5 model-embedding pairs showed consistent trends in terms of token importance. Tokens related to particular named entities, numbers, and domain-specific terminology were identified as having positive influence on the prediction of the Fact class. On the other hand, subjective language markers contributed to the classification of the instance as Non-Fact. Word importance summary gives a global overview of linguistic markers used by classifiers. Consistency of the identified important tokens among different models increases our belief that such markers are discriminative indeed.



Fig. 4. SVM + T5 pretrained

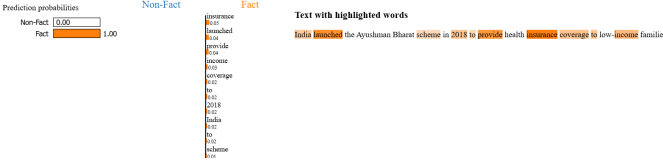


Fig. 5. SVM + T5 finetuned

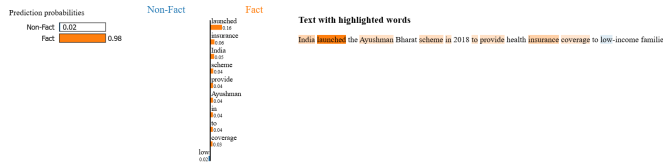


Fig. 6. SVM + SBERT



Fig. 7. SVM + BERT-FT

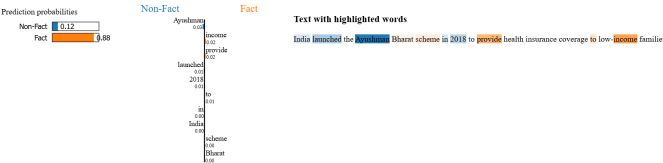


Fig. 8. LR + XLM-FT

## VII. CONCLUSION

In this paper, we have conducted an exhaustive exploration of the problem of classification of facts and non-facts within a wide range of embedding-classifier combinations. It was shown that length stratification during resampling is a crucial preprocessing procedure that eliminates the use of document length as a spurious feature by classifiers. Out of 77 examined embeddings-model combinations, the optimal result was obtained using SVM with T5 pre-trained embeddings (attention-masked mean pooling). Moreover, we have established that fine-tuning leads to better results when applied to BERT and XLM-RoBERTa embeddings, while pre-trained T5 embeddings can be used out-of-the-box. Interpretability analysis

based on LIME indicated the presence of consistent and linguistically meaningful features underlying the classification process in all top-performing models. For future research, one may consider ensembles of models trained using different embeddings, application of more powerful transformers such as RoBERTa-large and Flan-T5-large, as well as cross-domain generalization experiments. Extending the size of the dataset and adding multilingual fact-checking corpora is also advisable.

## REFERENCES

- [1] Sahu, I. and Majumdar, D., 2017, March. Detecting factual and non-factual content in news articles. In Proceedings of the 4th ACM IKDD Conferences on Data Sciences (pp. 1-12).
- [2] Schuster, T., Shah, D., Yeo, Y.J.S., Ortiz, D.R.F., Santus, E. and Barzilay, R., 2019, November. Towards debiasing fact verification models. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP) (pp. 3419-3425).
- [3] Alhindi, T., Petridis, S. and Muresan, S., 2018, November. Where is your evidence: Improving fact-checking by justification modeling. In Proceedings of the first workshop on fact extraction and verification (FEVER) (pp. 85-90).
- [4] Sahu, I., 2016. A study on detecting fact vs non-fact in news articles (Master's thesis, Indian Statistical Institute-Kolkata).
- [5] Wadden, D., Lin, S., Lo, K., Wang, L.L., van Zuylen, M., Cohan, A. and Hajishirzi, H., 2020, November. Fact or fiction: Verifying scientific claims. In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP) (pp. 7534-7550).
- [6] Sharifani, K., Amini, M., Akbari, Y. and Aghajanzadeh Godarzi, J., 2022. Operating machine learning across natural language processing techniques for improvement of fabricated news model. International Journal of Science and Information System Research, 12(9), pp.20-44.
- [7] Probiez, B., Stefański, P. and Kozak, J., 2021. Rapid detection of fake news based on machine learning methods. Procedia Computer Science, 192, pp.2893-2902.